Dropcheck: Product Brief

Name	**Henil Shah**
Project name	**Dropcheck**
Made for	**Airtribe GenAI Builder: "From Workflow Pain to a Working AI Product" (core project)**
Track	**Track 3: Marketing & Growth, Funnel Drop-off Diagnosis Tool**
Date	**30 January**

Submission links

Item	Link
Dropcheck page (live app)	https://claude.ai/artifact/8qcZS74zJutWY7zXg7YrEW
GitHub repository	https://github.com/henil-dev/dropcheck
Demo video	to be added

Built in Claude Code, versioned on GitHub, published as a Claude page. Claude is the AI layer. The live app needs a claude.ai sign-in for the AI diagnosis; the checks work without it.

---

1. Problem Statement

Growth managers watch funnels every week. When a step suddenly converts worse, the dashboard (Mixpanel, GA4, Amplitude) shows *where* users left, but it never says *why* or *what to try next*.

What usually happens next:

- The team spends a meeting guessing. The loudest opinion wins.
- The suggestions are vague ("improve the UX", "simplify onboarding"), so nobody can turn them into a test.
- Real causes get missed because nobody wrote down what changed that week: a new form field, a pricing change, a different ad campaign.
- Small steps with 40 users get as much attention as big steps losing 4,000.

The cost is time and lost conversions. A bad week often turns into a bad month because the first test the team runs is a guess.

The gap: nothing sits between "here is the funnel chart" and "here is a specific test we can launch on Monday".

2. User Persona

Priya, Growth Manager at a B2B SaaS startup (Series A, about 60 people)

- Owns the self-serve funnel: landing page → signup → onboarding → activation → paid.
- Reads the funnel every Monday and reports to the Head of Marketing.
- Comfortable with numbers and A/B testing, but has no data analyst of her own. The data team has a two-week queue.
- Has around 2 to 4 experiment slots per month, so a wasted test hurts.
- Her worry: "Leadership will ask why trial-to-paid dropped, and I will only have a guess."

What she needs from a tool: a quick, honest read of which step is actually broken, causes she can point to in her own data, and one experiment she can hand to a designer or engineer.

What she does not need: another dashboard, or a chat window that gives a different essay every time.

3. JTBD Statement

> **When** I see a sudden drop in conversion at one step of my funnel, **I want to** understand the most likely cause and what to test first, **so I can** fix the funnel quickly and defend my plan to leadership.

Trigger moment: a funnel report shows a sharp drop and nobody can explain it.

4. Why AI? (and where AI is not used)

Most of this problem does **not** need AI, so the product splits the work:

Part of the job	Done by	Why
Step conversion rates, drop size, users lost	Code	Simple math. An LLM can get arithmetic wrong.
Is this drop abnormal?	Code	Needs statistics (baseline comparison, z-test, sample size), not language.
Friction signals in step descriptions (payment, OTP, long form)	Code (keyword rules)	Cheap, predictable, explainable.
Experiment duration and sample size	Code	Standard power calculation.
Turning "Step 3 asks for a credit card, dropped 9 points after we added a phone field" into likely causes and testable hypotheses	**AI**	Needs reasoning across messy text: step descriptions, recent changes, product type. Rules cannot cover every product.
Writing a clear experiment brief	**AI**	Language work.
Checking whether AI output is generic or made up	Code	We do not trust the model to grade itself.

Why not just rules? There are thousands of possible funnels. A rule like "payment step → add trust badges" is exactly the generic advice users complain about. The value is in connecting *this* step description with *this* recent change and *this* drop. That is reasoning over unstructured text, which LLMs do well.

Why not just AI? If you paste a funnel into ChatGPT, it does the math in its head, treats a 40-user step like a 4,000-user step, and suggests "optimize the flow". The logic layer decides *what* to ask the model about and checks *what comes back*.

5. Feature List: Value vs Effort

#	Feature	Value	Effort	Decision
1	Structured funnel form (step name, description, users, optional baseline %)	High	Low	**Build**
2	Input validation (step counts can't increase, min 2 steps, description length)	High	Low	**Build**
3	Conversion math + users lost per step	High	Low	**Build**
4	Abnormal drop detection (baseline z-test, or compared to the other steps)	High	Medium	**Build**
5	Friction signal tagging from step descriptions	Medium	Low	**Build**
6	LLM diagnosis: reasons, 2–3 hypotheses, 1 experiment	High	Medium	**Build**
7	Generic-advice filter with one automatic retry	High	Medium	**Build** (main failure mode)
8	Evidence check: each reason must point at something in the input	High	Medium	**Build**
9	Confidence score (data quality + model confidence, with breakdown)	High	Low	**Build**
10	Experiment duration from traffic and expected lift	Medium	Low	**Build**
11	History of past analyses + "was this useful" feedback	Medium	Low	**Build**
12	Admin dashboard (usage, failures, retries, confidence, tokens)	Medium	Medium	**Build**
13	CSV upload / direct GA4 or Mixpanel import	High	High	Later
14	Segment breakdown (device, channel)	High	High	Later
15	Team accounts and sharing	Low (for now)	High	Skip

6. Failure Mode Analysis

Failure mode	What it looks like	Impact	How the product handles it
Generic advice (main risk)	"Improve UX", "streamline checkout", "build trust"	User stops trusting the tool after one use	Code scans every reason and hypothesis for a list of vague phrases and checks that it names a specific step element. If it fails, the prompt is retried once with the failing lines quoted back. Anything still generic is shown with a "generic" tag.
Made-up causes	"Your recent price increase caused this", when no price change was mentioned	Wrong test, wasted slot, lost credibility	Every reason must include an `evidence` field quoting the input. Code checks that the quote overlaps with the real input text. Unsupported reasons are tagged and lower confidence. The prompt says to label guesses as assumptions.
Diagnosing noise	A step with 35 users drops from 60% to 45%	Chasing random variation	Logic layer runs a z-test against the baseline and marks steps under 100 users as "low sample". Confidence is capped and the UI says so.

Failure mode	What it looks like	Impact	How the product handles it
Wrong math	Model miscalculates rates	Wrong focus step	The model never calculates. All numbers come from code and are passed in as facts.
Bad input	Step 3 has more users than Step 2; empty descriptions	Garbage in, garbage out	Validation blocks the request with a clear message before any API call.
LLM down / limit hit / bad JSON	Model error, usage limit, or broken output	Blank screen	JSON parse with a shape check, one retry with the error quoted back. If it still fails, the user still gets the full logic-layer report with a plain notice of what went wrong, and the failure is logged in the admin dashboard.
Viewer can't use Claude	Signed out, or declined the "allow Claude" prompt	Nothing happens	The checks run in the browser without any model, so the funnel table, significance tests and test sizing still show. The page says why the diagnosis is missing.
Over-confidence	Model says 95% sure on thin data	False certainty	Final confidence is the **lower** of a data-quality score and the model's own score, minus penalties for generic or unsupported lines.
Every step flagged	User doesn't know where to start	Noise	Only the top 2 abnormal steps are sent for diagnosis, ranked by users lost.

7. AI Trade-offs

- **Model choice:** Claude. The public version is a page published on claude.ai, and the AI step runs on Claude through that page. Claude follows long rule lists well, which matters here because the prompt carries nine rules about evidence and vague wording. The pipeline takes the model as a plug-in, so a local Node version of the same code runs on Gemini 2.5 Flash with no change to the checks.
- **Structured JSON vs free text:** JSON with a fixed shape makes the output checkable (evidence, generic filter, confidence), at the cost of less natural writing. Worth it. Claude gets the shape as text, so the parser also accepts JSON wrapped in a code fence.
- **Speed:** a Claude run takes 10–40 s because the model thinks before answering. Slower than a small model, but the tool is used once per funnel review, not in a live loop. The page shows each stage while it waits.
- **Retry on generic output:** improves quality but can double cost and latency for that request. Limited to one retry.
- **Narrow context:** only the abnormal steps plus surrounding steps are sent. Less context means less room for the model to wander, but it may miss a cause that started earlier in the funnel. Recent changes for the whole funnel are always included to reduce this.
- **No fine-tuning or RAG yet:** not enough real data to justify it. Stored analyses + feedback could become that dataset later.

8. Cost & Sustainability

How it is built and hosted: developed in Claude Code, versioned on GitHub, and published as a Claude page (claude.ai artifact). No separate server or hosting bill. The page keeps its history in its own built-in database.

Who pays for the model: each run uses the signed-in viewer's own Claude plan, and Claude asks the viewer to allow it the first time. So the cost to me as the builder is **$0 per analysis**, and usage can grow without a bill growing with it. The trade-off: someone without a Claude account only gets the logic-layer report.

If I ran it on a paid API instead (the local Node version with Gemini 2.5 Flash, at about $0.30 per 1M input tokens and $2.50 per 1M output tokens when written):

- One analysis ≈ 1,500 input + 900 output tokens ≈ **$0.0027**
- With a retry (15–20% of runs) ≈ $0.0055 for that run
- 1,000 analyses a month ≈ **$3–4**

Keeping it sustainable:

- Code does all the math, so the model only does reasoning.
- Validation stops bad input before any model call.
- Only flagged steps go to the model, not the whole funnel history.
- Identical inputs within five minutes reuse the stored answer instead of calling Claude again.
- Retries, failures and latency per run are stored and shown in the admin dashboard.
- If nothing stands out in the funnel, the model is not called at all.

Pricing idea if this were a product: free while it runs on the viewer's Claude plan. A team version on a paid API would be a small monthly plan. Model cost per analysis is under a cent.

9. Reflection

The biggest change in my thinking was how little of the product is actually AI. When I started, I pictured "paste your funnel, get insights". After mapping the workflow, most of the trust comes from boring parts: checking that the numbers make sense, saying when a sample is too small, and refusing vague answers.

The hardest design question was confidence. A single number from the model means nothing, because models are confident about everything. Splitting it into a data-quality score (which code can measure) and the model's own estimate, and taking the lower one, felt more honest.

What I would test with real growth managers:

1. Do they trust the "abnormal" flag, or do they want to set their own thresholds?
2. Is one experiment enough, or do they want a ranked list?
3. Does the "recent changes" field get filled in? It is optional, but the diagnosis is much better with it. If people skip it, the form needs to ask for it differently.

What I would build next: CSV import from GA4/Mixpanel exports, and comparing outcomes. If users record what happened after running the suggested experiment, the tool can learn which kinds of hypotheses actually work.